GUÍA DE ESTUDIO Y EXPOSICIÓN

Entender el código de un proyecto de fraude

Monitoring A Financial Fraud Detection Model
DataCamp 2104 · Python · NannyML

> **Objetivo:** poder explicar qué hace cada bloque, cómo se calculan las respuestas y qué conclusiones permiten sostener los datos.

DATOS
Referencia + producción
→
MONITORES
Exactitud + drift + media
→
RESPUESTAS
Meses + variable + importe

La idea central

El detector de fraude ya existe. Los archivos incluyen sus decisiones, sus probabilidades y las etiquetas reales. El proyecto construye **monitores** para detectar cuándo pierde rendimiento y qué cambios investigar. No entrena un detector nuevo.

Respuesta solicitada	Resultado aceptado
Meses con alerta estimada y real	april_2019, may_2019, june_2019
Variable más asociada al cambio	time_since_login_min
Media mensual que activa alerta	3069.8184 GBP, junio de 2019

Cómo estudiar esta guía

Primero comprende los datos (p. 2). Después sigue los doce pasos del código (pp. 3-10). Relaciona las respuestas con el diagnóstico y los controles (pp. 11-12). Practica la exposición (p. 13) y utiliza el código completo como referencia (p. 14).

Aprobación: DataCamp confirmó “Solución correcta” y otorgó 1500 XP el 9 de noviembre de 2026. El envío funcionó desde Edge con la misma solución que fallaba al enviarse desde el navegador integrado. La causa técnica exacta de esa diferencia no quedó determinada.

Los ejemplos señalados como didácticos ilustran el procedimiento; las tablas de resultados proceden de la ejecución sobre los CSV originales. Los decimales del código usan punto; la explicación utiliza coma decimal.

ANTES DEL CÓDIGO

Qué tenemos y qué queremos saber

El caso educativo presenta a Poundbank, un banco cuyo modelo de detección de fraude pierde exactitud después de su despliegue. Nuestra tarea consiste en detectar el deterioro, relacionarlo con cambios en los datos y comprobar el importe medio mensual.

Conjunto	Cobertura y tamaño	Papel
reference.csv	Enero-octubre de 2018 50.207 transacciones	Ajustar monitores y límites con datos anteriores.
analysis.csv	Noviembre de 2018-junio de 2019 39.967 transacciones	Medir y estimar lo que ocurre después.

Tres columnas que no debes confundir

Columna	Significado	Ejemplo didáctico
is_fraud	Lo que realmente ocurrió.	1: era fraude.
predicted_fraud	La decisión binaria del modelo.	0: predijo una operación legítima.
predicted_fraud_proba	Probabilidad de fraude emitida por el modelo.	0.30: asignó un 30 % de probabilidad.

En ese ejemplo el modelo comete un **falso negativo**: deja pasar una operación que realmente era fraudulenta. Un **falso positivo** ocurre cuando marca como fraude una operación legítima.

Las cinco variables de entrada

Variable	Qué describe
time_since_login_min	Minutos transcurridos desde el inicio de sesión.
transaction_amount	Importe de la operación en libras esterlinas.
transaction_type	Tipo de operación: CASH-OUT, PAYMENT, CASH-IN o TRANSFER.
is_first_transaction	Si es la primera transacción del usuario.
user_tenure_months	Antigüedad del usuario en meses.

> **La fecha**, timestamp, permite formar bloques mensuales. Las cinco variables anteriores se usan para analizar drift; las predicciones y etiquetas se usan para medir rendimiento.

La referencia no es aquí el conjunto de entrenamiento del detector. El modelo entrenado y sus datos de entrenamiento no están disponibles. Se estudian sus predicciones ya calculadas.

PASOS 1 Y 2

Importar y cargar los datos

```
from pathlib import Path
import calendar
import numpy as np
import pandas as pd
import nannyml as nml
nml.disable_usage_logging()
np.random.seed(42)
```

Herramienta	Para qué se utiliza
Path	Localizar archivos y construir rutas.
calendar	Obtener los nombres de los meses.
numpy (np)	Operaciones numéricas y semilla aleatoria.
pandas (pd)	Leer CSV y trabajar con tablas (DataFrames).
nannyml (nml)	Estimar rendimiento, detectar drift y generar alertas.

disable_usage_logging() desactiva la telemetría de NannyML. **np.random.seed(42)** favorece la reproducibilidad de los procedimientos que utilicen aleatoriedad; no establece las respuestas ni garantiza identidad entre todas las versiones del software.

```
data_dir = Path('.') if Path('reference.csv').exists() else Path('data')
reference = pd.read_csv(
    data_dir / 'reference.csv', parse_dates=['timestamp']
)
analysis = pd.read_csv(
    data_dir / 'analysis.csv', parse_dates=['timestamp']
)
```

La primera línea utiliza la carpeta actual si encuentra reference.csv; en otro caso busca en data. El operador **/** entre objetos Path construye una ruta, no divide números.

read_csv() crea una tabla de pandas. **parse_dates** interpreta timestamp como fecha para agrupar después por mes. reference y analysis son nombres de variables que guardan esas tablas.

> **Para exponer:** “Primero cargué dos periodos separados. El pasado define la referencia; los meses posteriores permiten evaluar si el comportamiento cambia.”

PASO 3

Configurar los monitores

```
parameters = dict(
    timestamp_column_name='timestamp',
    y_true='is_fraud',
    y_pred='predicted_fraud',
    y_pred_proba='predicted_fraud_proba',
    problem_type='classification_binary',
    metrics=['accuracy'],
    chunk_period='M'
)
```

Este **diccionario** reúne la configuración compartida por el estimador y el calculador del rendimiento. Permite mantener los nombres de columnas y la periodicidad consistentes.

Parámetro	Lectura en palabras
timestamp_column_name	Usa timestamp para ordenar y agrupar el tiempo.
y_true	La verdad conocida está en is_fraud.
y_pred	La decisión del modelo está en predicted_fraud.
y_pred_proba	La probabilidad está en predicted_fraud_proba.
classification_binary	Hay dos clases: fraude y no fraude.
metrics=['accuracy']	Mide la proporción total de decisiones correctas.
chunk_period='M'	Forma bloques mensuales: 10 de referencia y 8 de producción.

Qué es accuracy

> **Exactitud = número de predicciones correctas / número total de transacciones.**
> Ejemplo didáctico: 940 aciertos entre 1.000 operaciones dan 0,94, es decir, 94 %.

Una exactitud alta no explica qué tipos de errores se cometen. Por eso el análisis ampliado también calcula precisión, recall y falsos negativos. En estos datos las clases están aproximadamente equilibradas; eso no representa necesariamente el fraude bancario real.

Qué significa **parameters

Cuando aparece **nml.CBPE(**parameters)**, los dos asteriscos desempaquetan el diccionario como argumentos con nombre. Es equivalente a escribir dentro de la llamada cada parámetro anterior.

PASO 4 · CBPE

Estimar sin conocer la respuesta real

```
cbpe = nml.CBPE(**parameters).fit(reference)
estimated = cbpe.estimate(
    analysis.drop(columns='is_fraud')
)
```

CBPE significa Confidence-Based Performance Estimation: estimación del rendimiento basada en las probabilidades del modelo. Resulta útil cuando las etiquetas reales llegan con retraso.

Qué hace fit(reference)

Ajusta el monitor con el periodo de referencia. Usa las probabilidades y las etiquetas conocidas para evaluar su calibración y preparar la estimación y sus límites. En este caso, NannyML determinó que era necesario calibrar las probabilidades.

> **Ajustar el monitor no es reentrenar el detector.** Se conserva la decisión original predicted_fraud. Calibrar probabilidades para CBPE no cambia esas decisiones suministradas.

Ejemplo didáctico del cálculo de exactitud estimada

Con una probabilidad calibrada de fraude p, la probabilidad estimada de acertar es p si la decisión es fraude, y 1 - p si la decisión es no fraude.

Operación	p calibrada	Decisión	Acierto esperado
A	0,80	Fraude	0,80
B	0,20	No fraude	1 - 0,20 = 0,80
C	0,60	No fraude	1 - 0,60 = 0,40

```
Exactitud estimada del ejemplo = (0.80 + 0.80 + 0.40) / 3
                               = 0.6667 = 66.67 %
```

Este ejemplo explica el mecanismo; no son transacciones extraídas del proyecto. En el caso real se promedian los aciertos esperados de cada mes.

Por qué eliminamos is_fraud

analysis.drop(columns='is_fraud') entrega al estimador una copia sin la etiqueta real de producción. Evita usar las respuestas que estamos intentando anticipar. Como no se usa inplace=True, analysis conserva la columna para la validación posterior.

La estimación depende de que las probabilidades calibradas sigan siendo informativas. Si cambia la relación entre datos y etiquetas, CBPE puede dejar de estimar bien. Por eso la contrastamos con el rendimiento observado.

PASOS 5 Y 6

Medir y preparar la comparación

```
realized = (
    nml.PerformanceCalculator(**parameters)
    .fit(reference)
    .calculate(analysis)
)
```

PerformanceCalculator compara predicted_fraud con is_fraud para calcular la proporción real de aciertos. fit establece la referencia; calculate evalúa la producción, donde sí están disponibles las etiquetas verdaderas.

Objeto	Qué representa	¿Usa la verdad de producción?
estimated	Exactitud estimada con probabilidades.	No.
realized	Exactitud observada comparando decisión y etiqueta.	Sí.

Límites de alerta obtenidos

Los límites aprendidos fueron **93,6367 %** y **95,0808 %**. Salir de ese intervalo activa una alerta. Una subida inusual también puede activarla: alerta no es sinónimo automático de deterioro.

```
e = estimated.filter(period='analysis').to_df()
r = realized.filter(period='analysis').to_df()
assert (
    e[('chunk', 'key')].tolist()
    == r[('chunk', 'key')].tolist()
)
```

filter(period='analysis') conserva los ocho meses de producción. **to_df()** convierte el resultado de NannyML en una tabla de pandas. Los nombres e y r son abreviaturas de estimado y real.

Columna de dos niveles	Contenido
('chunk', 'key')	Identificador del mes, por ejemplo 2019-04.
('accuracy', 'value')	Valor numérico de exactitud, entre 0 y 1.
('accuracy', 'alert')	Booleano: True si hay alerta y False si no.

La pareja entre paréntesis es una **tupla** que identifica una columna con dos niveles. El assert comprueba que los meses coinciden en ambas tablas y en el mismo orden. Si no coinciden, detiene el programa en lugar de comparar meses incorrectos.

PASO 7 · PRIMERA RESPUESTA

Obtener los meses con ambas alertas

```
joint = (
    e[('accuracy', 'alert')].to_numpy()
    & r[('accuracy', 'alert')].to_numpy()
)
```

to_numpy() obtiene arreglos de booleanos. El operador **&** aplica un "y" lógico por posición: solo True & True produce True. Se necesita la intersección de las dos alertas, no su unión.

Mes	Estimada %	Real %	Est.	Real	Ambas
2018-11	94.195	94.268	No	No	No
2018-12	94.325	94.100	No	No	No
2019-01	94.545	95.176	No	Sí	No
2019-02	94.461	94.312	No	No	No
2019-03	94.374	94.004	No	No	No
2019-04	91.105	91.463	Sí	Sí	Sí
2019-05	91.024	91.541	Sí	Sí	Sí
2019-06	91.166	91.423	Sí	Sí	Sí

Enero no entra: su exactitud real supera el límite superior, pero la estimada no activa alerta. Abril, mayo y junio están por debajo del límite inferior en ambas medidas.

```
months_with_performance_alerts = [
    f'{calendar.month_name[pd.Period(key).month].lower()}_'
    f'{pd.Period(key).year}'
    for key in e.loc[joint, ('chunk', 'key')]
]
```

loc selecciona las filas donde joint es verdadero y extrae el mes. **pd.Period** permite obtener mes y año; calendar obtiene el nombre en inglés y lower lo pasa a minúsculas. Las dos cadenas f contiguas forman una sola cadena.

> **Ejemplo de formato:** 2019-04 → 4 → April → april → april_2019.
> **Resultado:** ['april_2019', 'may_2019', 'june_2019'].

Preparar y medir cambios en los datos

```
features = [
    'time_since_login_min', 'transaction_amount',
    'transaction_type', 'is_first_transaction',
    'user_tenure_months'
]
drift_reference = reference.copy()
drift_analysis = analysis.copy()
for frame in [drift_reference, drift_analysis]:
    frame['transaction_type'] = (
        frame.transaction_type.fillna('__MISSING__')
    )
```

Un **drift** es un cambio en la distribución de los datos. copy crea tablas independientes. fillna representa la ausencia de tipo de transacción como una categoría explícita: no adivina un tipo ni elimina operaciones.

```
drift = nml.UnivariateDriftCalculator(
    column_names=features,
    timestamp_column_name='timestamp',
    chunk_period='M',
    continuous_methods=['kolmogorov_smirnov'],
    categorical_methods=['chi2'],
    treat_as_categorical=[
        'transaction_type', 'is_first_transaction'
    ]
).fit(drift_reference).calculate(drift_analysis)
```

Método	Variables	Qué compara
Kolmogorov-Smirnov (KS)	Tiempo desde login, importe y antigüedad.	Distribuciones de valores numéricos.
Chi-cuadrado (chi2)	Tipo de operación y primera transacción.	Frecuencias de las categorías.

Univariante significa analizar cada variable por separado. is_first_transaction se trata como categórica porque representa dos situaciones, no una magnitud continua. fit establece la distribución de referencia; calculate compara cada mes posterior con ella.

> **Ejemplo:** si antes abundaban las operaciones inmediatamente después del login y luego casi desaparecen, la distribución cambia. Eso no demuestra todavía que el cambio cause los errores del modelo.

PASO 10 · SEGUNDA RESPUESTA

Ordenar las variables por asociación

```
ranker = nml.CorrelationRanker().fit(
    realized.filter(period='reference')
)
ranking = ranker.rank(
    drift.filter(period='analysis'),
    realized.filter(period='analysis'),
    only_drifting=True
)
highest_correlation_feature = str(
    ranking.iloc[0].column_name
)
```

El ranker obtiene la exactitud media de los meses de referencia. Para cada mes de producción calcula la desviación absoluta respecto a esa media:

```
cambio_m = abs(accuracy_m - media_accuracy_referencia)
```

Para cada variable compara dos series de ocho valores: la magnitud del drift mensual y el cambio absoluto de exactitud mensual. Calcula la correlación de Pearson y ordena las variables.

Variable con drift	Pearson r	Posición relativa
time_since_login_min	0,9704	Primera
transaction_amount	0,5458	Segunda

only_drifting=True omite variables que no activaron ninguna alerta de drift. **iloc[0]** toma la primera fila; **column_name** extrae su nombre y str lo convierte a texto.

> **Resultado:** highest_correlation_feature = 'time_since_login_min'.
> Los meses con mayor cambio en esa distribución tendieron a presentar mayor desviación del rendimiento habitual.

Cómo interpretar sin exagerar

El ranking no mide la importancia interna de las variables del detector. Tampoco correlaciona los valores individuales de login con la etiqueta de fraude. Relaciona **cambios mensuales de distribución** con **cambios mensuales de rendimiento**.

Al usar una diferencia absoluta, el método puede asociarse tanto con mejoras como con deterioros. La dirección se interpreta mirando las métricas: en abril-junio hay deterioro. Una correlación alta no demuestra causalidad y ocho meses son una muestra pequeña.

PASOS 11 Y 12 · TERCERA RESPUESTA

Encontrar el importe anómalo y mostrarlo

```
average = nml.SummaryStatsAvgCalculator(
    column_names=['transaction_amount'],
    timestamp_column_name='timestamp',
    chunk_period='M'
).fit(reference).calculate(analysis)
a = average.filter(period='analysis').to_df()
alerted_amounts = a.loc[
    a[('transaction_amount', 'alert')],
    ('transaction_amount', 'value')
]
assert len(alerted_amounts) == 1
alert_avg_transaction_amount = round(
    float(alerted_amounts.iloc[0]), 4
)
```

La media mensual es la suma de los importes del mes dividida entre su número de transacciones. Incluye operaciones fraudulentas y legítimas. El monitor ajusta sus límites con las medias mensuales de referencia; la estrategia predeterminada usa su media más o menos tres desviaciones estándar.

Concepto	Valor obtenido
Límite inferior	£2908,4340
Límite superior	£3021,4111
Media de junio de 2019	£3069,818355...
Exceso sobre el límite superior	Aproximadamente £48,41

loc[filas, columna] conserva las medias de los meses con alerta. Solo hay una: junio. El assert verifica esa condición para no seleccionar arbitrariamente un mes si hubiera varios. float produce un número decimal estándar; round(..., 4) lo redondea a cuatro decimales.

> **3069.8184 es la media del mes con alerta, no el umbral.** El límite superior que se supera es 3021.4111 aproximadamente.

```
print('months_with_performance_alerts =', months_with_performance_alerts)
print('highest_correlation_feature =', highest_correlation_feature)
print('alert_avg_transaction_amount =', alert_avg_transaction_amount)
```

print solo muestra las variables ya calculadas. Las respuestas no se escribieron a mano para que pasaran el corrector: se extrajeron de los resultados de los monitores. El código completo del anexo conserva también el mensaje explicativo del assert.

DEL CÁLCULO A LA CONCLUSIÓN

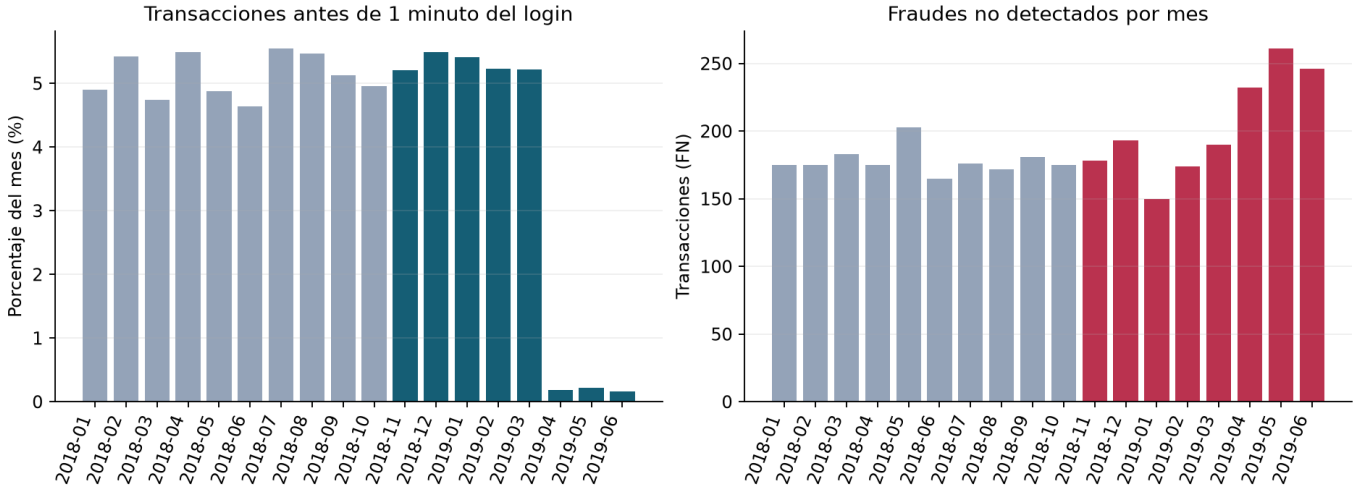
Qué significan los resultados

Indicador	Nov. 2018-mar. 2019	Abr.-jun. 2019
Transacciones del periodo	24.938	15.029
Exactitud agregada	94,37 %	91,48 %
Tasa de error	5,63 %	8,52 %
Operaciones antes de 1 minuto del login	5,31 %	0,19 %
Fraudes no detectados / fraudes reales	7,12 %	9,82 %

Las tasas agregadas se calculan sobre todas las transacciones de cada periodo. No son un promedio sin ponderar de porcentajes mensuales. Los periodos tienen distinta duración: compara tasas antes que totales brutos.

```
Caida de exactitud: 94.3740 % - 91.4765 % = 2.90 puntos porcentuales
Aumento relativo del error:
(0.085235 / 0.056260 - 1) * 100 = aproximadamente 51.5 %
```

Una caída de 2,90 puntos en exactitud puede parecer pequeña, pero implica aproximadamente **51,5 % más errores en términos relativos**. No significa que la exactitud haya caído 51,5 puntos.



Izquierda: porcentaje mensual de operaciones antes de un minuto del login. Derecha: número mensual de fraudes no detectados. Las barras describen el cambio; no prueban por sí solas su causa.

Conclusión que sí podemos sostener

Hay deterioro simultáneo estimado y observado en abril-junio; el cambio del tiempo desde el login es la principal señal a investigar; la media de los importes es anómala en junio. El descenso de operaciones rápidas aparece tanto en fraude como en operaciones legítimas.

Siguiente paso: verificar la captura del login, cambios en la aplicación y composición de usuarios; revisar errores por segmento y evaluar ajustes con datos futuros. No se ha demostrado un ataque adaptativo ni una recuperación del detector.

Cómo se verificó el análisis completo

codigo_datacamp.py resuelve las tres preguntas. src/analisis.py amplía el trabajo para producir el informe, las tablas, las gráficas y las comprobaciones independientes.

Función	Responsabilidad
load_data()	Valida columnas, fechas, clases, probabilidades, nulos, duplicados y separación temporal; registra hashes SHA-256.
monthly_metrics()	Calcula accuracy, precisión, recall, F1, AUC, AP, calibración, matrices de confusión e importes por mes.
run_monitoring()	Ejecuta CBPE, rendimiento real, drift y medias; guarda resultados y compara con cálculos independientes.
robustness()	Calcula Pearson y Spearman, retira un mes cada vez y ajusta múltiples contrastes.
additional_evidence()	Compara los periodos estable y de alerta; analiza segmentos y el umbral de decisión suministrado.
make_figures()	Genera las gráficas de rendimiento y diagnóstico.
run()	Organiza la primera prueba y la versión final y registra versiones.

Por qué confiamos en los cálculos

La exactitud observada se contrastó con scikit-learn y las medias con pandas, con tolerancias de 1e-12. Se comprobó que los meses estaban alineados, que CBPE no recibía etiquetas de producción y que la primera prueba y la versión final daban las mismas respuestas.

La revisión hizo explícitos los nulos de transaction_type y limitó el ranking principal a los ocho meses de producción. El Pearson del login fue 0,9704; al retirar un mes quedó entre 0,961 y 0,988. Sin embargo, Spearman fue 0,5238 (p = 0,1827): la asociación parece reflejar sobre todo un cambio de régimen, no una relación monotónica demostrada.

No cambiar las decisiones originales

El análisis adicional encontró que el umbral 0,65 reproduce exactamente las etiquetas predicted_fraud suministradas. Usar 0,50 cambiaría decisiones y respondería otra pregunta. El proyecto conserva las etiquetas originales; no optimiza un umbral nuevo.

Aprobación y alcance: DataCamp aceptó las tres respuestas y otorgó 1500 XP. Esa aprobación valida el ejercicio, no demuestra causalidad ni certifica el detector para producción. El entorno local usó NannyML 0.13.1; DataLab reprodujo las respuestas con 0.12.1.

PRÁCTICA DE PRESENTACIÓN

Guion para exponer y repasar

Guion de aproximadamente dos minutos

“Analicé un modelo de fraude que ya tenía predicciones. Mi objetivo fue comprobar si seguía funcionando bien y priorizar los cambios que debían investigarse.”

“Separé los datos en referencia y producción. La referencia cubre enero-octubre de 2018 y permite ajustar los monitores. La producción cubre noviembre de 2018-junio de 2019 y se evaluó en bloques mensuales.”

“Primero estimé la exactitud con CBPE, sin entregarle las etiquetas reales de producción. Después calculé la exactitud observada con esas etiquetas. La intersección de las alertas identificó abril, mayo y junio de 2019.”

“Luego comparé las distribuciones de cinco variables mediante KS y chi-cuadrado. Relacioné el drift mensual con el cambio absoluto de rendimiento. La variable con mayor asociación fue el tiempo desde el inicio de sesión, con Pearson de 0,9704.”

“También comprobé los importes medios mensuales. Junio tuvo una media de 3069,8184 libras, por encima del límite superior de 3021,41 libras. Verifiqué los cálculos y DataCamp aprobó las respuestas.”

“La exactitud pasó del 94,37 % al 91,48 %. Detectamos y caracterizamos el deterioro, pero no demostramos su causa ni reentrenamos el detector. Recomendaría verificar la captura de datos y evaluar mejoras con datos futuros.”

Preguntas que conviene saber responder

Pregunta	Respuesta breve
¿Por qué no se incluye enero?	Solo tiene alerta en la exactitud real; se requieren ambas.
¿Qué diferencia hay entre fit y calculate?	fit ajusta la referencia del monitor; calculate evalúa datos posteriores.
¿Por qué quitar is_fraud al estimar?	Para estimar sin consultar la verdad de producción.
¿Drift significa necesariamente fallo?	No. Un cambio de distribución puede no perjudicar el modelo.
¿3069,8184 es el umbral?	No: es la media de junio. El umbral superior es 3021,4111.
¿Qué demuestra la correlación?	Asociación entre cambios mensuales; no causalidad.

Autoevaluación: si la alerta estimada es True y la real es False, joint es False. Si la media fuera £3000, no superaría el límite superior. Si hubiera dos medias con alerta, el assert detendría la selección de un único importe.

ANEXO · TEXTO ORIGINAL DEL ARCHIVO

Código completo de referencia

Código íntegro de codigo_datacamp.py. Ejecutarlo requiere pandas, NumPy y NannyML, además de ambos CSV. En el notebook de DataCamp, la instalación de NannyML está en una celda previa. Los bloques anteriores solo redistribuyen líneas para facilitar la lectura.

```
"""Solucion para ejecutar en una celda de DataCamp; respuestas calculadas, no prefijadas."""
from pathlib import Path
import calendar
import numpy as np
import pandas as pd
import nannyml as nml
nml.disable_usage_logging()
np.random.seed(42)
data_dir=Path('.') if Path('reference.csv').exists() else Path('data')
reference=pd.read_csv(data_dir/'reference.csv',parse_dates=['timestamp'])
analysis=pd.read_csv(data_dir/'analysis.csv',parse_dates=['timestamp'])
parameters=dict(timestamp_column_name='timestamp',y_true='is_fraud',
    y_pred='predicted_fraud',y_pred_proba='predicted_fraud_proba',
    problem_type='classification_binary',metrics=['accuracy'],chunk_period='M')
cbpe=nml.CBPE(**parameters).fit(reference)
estimated=cbpe.estimate(analysis.drop(columns='is_fraud'))
realized=nml.PerformanceCalculator(**parameters).fit(reference).calculate(analysis)
e=estimated.filter(period='analysis').to_df()
r=realized.filter(period='analysis').to_df()
assert e[('chunk','key')].tolist()==r[('chunk','key')].tolist()
joint=e[('accuracy','alert')].to_numpy() & r[('accuracy','alert')].to_numpy()
months_with_performance_alerts=[
    f'{calendar.month_name[pd.Period(key).month].lower()}_{pd.Period(key).year}'
    for key in e.loc[joint,('chunk','key')]]
features=['time_since_login_min','transaction_amount','transaction_type',
          'is_first_transaction','user_tenure_months']
drift_reference=reference.copy()
drift_analysis=analysis.copy()
for frame in [drift_reference,drift_analysis]:
    frame['transaction_type']=frame.transaction_type.fillna('__MISSING__')
drift=nml.UnivariateDriftCalculator(column_names=features,
    timestamp_column_name='timestamp',chunk_period='M',
    continuous_methods=['kolmogorov_smirnov'],categorical_methods=['chi2'],
    treat_as_categorical=['transaction_type','is_first_transaction']
    ).fit(drift_reference).calculate(drift_analysis)
ranker=nml.CorrelationRanker().fit(realized.filter(period='reference'))
ranking=ranker.rank(drift.filter(period='analysis'),realized.filter(period='analysis'),only_drifting=True)
highest_correlation_feature=str(ranking.iloc[0].column_name)
average=nml.SummaryStatsAvgCalculator(column_names=['transaction_amount'],
    timestamp_column_name='timestamp',chunk_period='M').fit(reference).calculate(analysis)
a=average.filter(period='analysis').to_df()
alerted_amounts=a.loc[a[('transaction_amount','alert')],('transaction_amount','value')]
assert len(alerted_amounts)==1, 'Revisar los meses con alerta antes de asignar un unico importe'
alert_avg_transaction_amount=round(float(alerted_amounts.iloc[0]),4)
print('months_with_performance_alerts =',months_with_performance_alerts)
print('highest_correlation_feature =',highest_correlation_feature)
print('alert_avg_transaction_amount =',alert_avg_transaction_amount)
```

Materiales del proyecto: codigo_datacamp.py; src/analisis.py; output/final/performance_comparison.csv; output/final/answers.json; output/final/phase_summary.csv; evidence/envio_datacamp_estado.json.

Enunciado del proyecto · Cuaderno aprobado en DataLab